



Ad Soyad: _____

Numara: _____

Tarih: _____

1 Çalışma 1: Event Gözlemcisi

Örnek Kod

Aşağıdaki programı `olay_gozlemci.py` dosyasına yazın ve çalıştırın:

```
1 import pygame
2 import sys
3
4 pygame.init()
5
6 GENISLIK = 800
7 YUKSEKLIK = 600
8
9 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Olay Gözlemci")
11 saat = pygame.time.Clock()
12
13 KOYU_GRI = (40, 40, 40)
14
15 çalıştır = True
16 while çalıştır:
17     for olay in pygame.event.get():
18         if olay.type == pygame.QUIT:
19             çalıştır = False
20         else:
21             # Tüm olayları konsola yazdır
22             print(f"Olay: {olay}")
23
24     ekran.fill(KOYU_GRI)
25     pygame.display.flip()
26     saat.tick(60)
27
28 pygame.quit()
29 sys.exit()
```

`event.get()` Her Turda Çağrılmalı

`pygame.event.get()` fonksiyonu olay kuyruğunu boşaltır ve bekleyen tüm olayları bir liste olarak döndürür. Her oyun döngüsünde mutlaka çağrılmalıdır. Çağrılmadığı takdirde kuyruk dolar ve pencere yanıt vermez hale gelir.

Talep 1: Olayları Gözlemle

Kodu çalıştırın, farklı tuşlara basın ve fare ile tıklayın. Terminalde gözlemlenen olay türlerini aşağıya yazın:

Klavyeye basınca	
Klavyeyi bırakınca	
Fare tıklayınca	
Fare hareket edince	
Pencere kapatılınca	

Talep 2: Olay Filtresi

Olay işleme bloğunu düzenleyerek yalnızca `KEYDOWN` ve `MOUSEBUTTONDOWN` olaylarını yazdırın, diğerlerini göz ardı edin.

İpucu: `if` ifadesiyle olay türünü kontrol edin:

```
1 if olay.type == pygame.KEYDOWN:
2     print(f"Basıldı: {pygame.key.name(olay.key)}")
3 elif olay.type == pygame.MOUSEBUTTONDOWN:
4     print(f"Fare: dugme={olay.button}, konum={olay.pos}")
```

2 Çalışma 2: Klavye ile Hareket

Örnek Kod

Aşağıdaki programı `klavye_hareket.py` dosyasına yazın ve çalıştırın:

```
1 import pygame
2 import sys
3
4 pygame.init()
5
6 GENISLIK = 800
7 YUKSEKLIK = 600
8
9 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Klavye ile Hareket")
11 saat = pygame.time.Clock()
12 FPS = 60
13
14 LACIVERT = (20, 30, 70)
15 SARI = (255, 220, 50)
16
17 # Kare özellikleri
18 kare_x = GENISLIK // 2
19 kare_y = YUKSEKLIK // 2
20 kare_bov = 40
21 HIZ = 200 # piksel/saniye
22
23 çalıştır = True
24 while çalıştır:
25     dt = saat.tick(FPS) / 1000.0 # saniye cinsinden delta time
26
```

```

27     for olay in pygame.event.get():
28         if olay.type == pygame.QUIT:
29             çalıştır = False
30         elif olay.type == pygame.KEYDOWN:
31             if olay.key == pygame.K_ESCAPE:
32                 çalıştır = False
33
34         # Sürekli girdi kontrolü (get_pressed)
35         tuslar = pygame.key.get_pressed()
36         if tuslar[pygame.K_UP]:
37             kare_y -= HIZ * dt
38         if tuslar[pygame.K_DOWN]:
39             kare_y += HIZ * dt
40         if tuslar[pygame.K_LEFT]:
41             kare_x -= HIZ * dt
42         if tuslar[pygame.K_RIGHT]:
43             kare_x += HIZ * dt
44
45         ekran.fill(LACIVERT)
46         pygame.draw.rect(ekran, SARI,
47                          (int(kare_x), int(kare_y), kare_boy, kare_boy))
48         pygame.display.flip()
49
50     pygame.quit()
51     sys.exit()

```

Sürekli Girdi vs. Ayrık Girdi Farkı

İki farklı girdi okuma yöntemi vardır:

- `pygame.key.get_pressed()` → **Sürekli girdi**: Tuş basılı tutulduğu sürece her karede `True` döner. Hareket için idealdir.
- `KEYDOWN` olayı → **Ayrık girdi**: Tuşa yalnızca bir kez basıldığında tetiklenir. Zıplama, ateş etme gibi tek seferlik eylemler için kullanın.

Talep 1: Hız Deneyi

`HIZ` sabitini sırasıyla 100, 300 ve 500 olarak değiştirip programı çalıştırın. Her hız değerindeki davranış farkını gözlemleyin ve aşağıya not edin:

HIZ = 100	
HIZ = 300	
HIZ = 500	

Talep 2: WASD Desteği

Ok tuşlarının yanı sıra WASD klavye düzenini de destekleyin:

- `K_w` → Yukarı
- `K_a` → Sol
- `K_s` → Aşağı
- `K_d` → Sağ

Talep 3: Renk Değişimi

Space tuşuna basıldığında karenin rengi değişsin. `KEYDOWN` olayını kullanarak her basışta farklı bir renk seçin:

```
1 renkler = [(255,220,50), (50,200,100), (50,150,255), (255,100,100)]
2 renk_indis = 0
3
4 # KEYDOWN olayında:
5 if olay.key == pygame.K_SPACE:
6     renk_indis = (renk_indis + 1) % len(renkler)
```

3 Çalışma 3: Fare ile Etkileşim

Örnek Kod

Aşağıdaki programı `fare_etiklesim.py` dosyasına yazın ve çalıştırın:

```
1 import pygame
2 import sys
3
4 pygame.init()
5
6 GENISLIK = 800
7 YUKSEKLIK = 600
8
9 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Fare ile Etkilesim")
11 saat = pygame.time.Clock()
12
13 KOYU_MAVI = (15, 25, 60)
14 MAVI = (50, 120, 255)
15
16 daireler = [] # (x, y) pozisyonlarini tut
17
18 çalıştır = True
19 while çalıştır:
20     for olay in pygame.event.get():
21         if olay.type == pygame.QUIT:
22             çalıştır = False
23         elif olay.type == pygame.MOUSEBUTTONDOWN:
24             if olay.button == 1: # Sol tikla
25                 daireler.append(olay.pos)
```

```

26
27     ekran.fill(KOYU_MAVI)
28
29     for konum in daireler:
30         pygame.draw.circle(ekran, MAVI, konum, 20)
31
32     pygame.display.flip()
33     saat.tick(60)
34
35 pygame.quit()
36 sys.exit()

```

Talep 1: Tıklama Rengi

Sol tıklama mavi((50, 120, 255)),sağ tıklama kırmızı((220, 60, 60)) daire oluştursun. Listeye renk bilgisini de ekleyin:

```

1 # (x, y, renk) uclusu olarak sakla
2 daireler.append((olay.pos[0], olay.pos[1], MAVI))

```

Sağ tıklama için `olay.button == 3` koşulunu kullanın.

Talep 2: Fareyi Takip Eden Nesne

`pygame.mouse.get_pos()` ile fare konumunu her karede okuyun ve fareyi takip eden küçük bir daire çizin:

```

1 # Güncelleme ve çizim bölümünde:
2 fare_x, fare_y = pygame.mouse.get_pos()
3 pygame.draw.circle(ekran, (255, 255, 100),
4     (fare_x, fare_y), 12)

```

Talep 3: Daireleri Temizle

`K_c` tuşuna basıldığında tüm daireler silinsin. `KEYDOWN` olayında listeyi boşaltın:

```

1 if olay.type == pygame.KEYDOWN:
2     if olay.key == pygame.K_c:
3         daireler.clear()

```

4 Çalışma 4: Sınır Kontrolü

Örnek Kod

Aşağıdaki programı `sinir_kontrol.py` dosyasına yazın ve çalıştırın:

```

1 import pygame
2 import sys
3
4 pygame.init()
5
6 GENISLIK = 800

```

```

7  YUKSEKLIK = 600
8
9  ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Sinir Kontrolu - Clamp")
11 saat = pygame.time.Clock()
12 FPS = 60
13
14 LACIVERT = (15, 25, 60)
15 TURUNCU = (255, 160, 20)
16
17 # Top özellikleri
18 top_x = 100.0
19 top_y = 100.0
20 top_r = 25          # yarıçap
21 hiz_x = 250.0       # piksel/saniye
22 hiz_y = 180.0
23
24 çalıştır = True
25 while çalıştır:
26     dt = saat.tick(FPS) / 1000.0
27
28     for olay in pygame.event.get():
29         if olay.type == pygame.QUIT:
30             çalıştır = False
31
32     # Hareketi güncelle
33     top_x += hiz_x * dt
34     top_y += hiz_y * dt
35
36     # Clamp: sinir içerisinde tut
37     top_x = max(top_r, min(top_x, GENISLIK - top_r))
38     top_y = max(top_r, min(top_y, YUKSEKLIK - top_r))
39
40     ekran.fill(LACIVERT)
41     pygame.draw.circle(ekran, TURUNCU,
42                        (int(top_x), int(top_y)), top_r)
43     pygame.display.flip()
44
45 pygame.quit()
46 sys.exit()

```

Üç Sınır Stratejisi

Bir nesne ekranın kenarına ulaştığında üç farklı strateji uygulanabilir:

- **Clamp (Kilitleme):** Nesneyi kenarda durdur. `x = max(r, min(x, GEN - r))`
- **Bounce (Sekme):** Kenara çarpınca hız tersine döner. `if x > GEN - r: hiz_x = -hiz_x`
- **Wrap (Sarmalama):** Bir kenardan çıkan nesne karşı kenardan geri girer. `if x > GEN: x = -r`

Talep 1: Bounce Stratejisi

Clamp yerine Bounce (sekme) stratejisini uygulayın. Top duvara çarptığında ilgili hız bileşenini tersine çevirin:

```
1 if top_x - top_r < 0 or top_x + top_r > GENISLIK:
2     hiz_x = -hiz_x
3 if top_y - top_r < 0 or top_y + top_r > YUKSEKLIK:
4     hiz_y = -hiz_y
```

Talep 2: Wrap Stratejisi

Wrap (sarmalama) stratejisini uygulayın. Top sağ kenardan çıkınca sol kenardan, alt kenardan çıkınca üst kenardan tekrar girsin:

```
1 if top_x - top_r > GENISLIK:
2     top_x = -top_r
3 elif top_x + top_r < 0:
4     top_x = GENISLIK + top_r
5 if top_y - top_r > YUKSEKLIK:
6     top_y = -top_r
7 elif top_y + top_r < 0:
8     top_y = YUKSEKLIK + top_r
```

Talep 3: Üç Top

Üç topu aynı anda farklı stratejilerle hareket ettirin:

- **Turuncu top:** Clamp – kenarda durur
- **Mavi top:** Bounce – sekme yapar
- **Yeşil top:** Wrap – karşı kenardan girer

Her top için ayrı `x`, `y`, `hiz_x`, `hiz_y` değişkenleri tanımlayın.

Bonus: Mini Yılan Prototipi

Öğrendiklerinizi birleştirerek yılan oyununun basit bir prototipini yazın:

Kurallar:

- Izgara tabanlı hareket – yılan her adımda bir hücre ilerler
- Ok tuşlarıyla yön değiştirme (KEYDOWN ile)
- Kırmızı bir yem rastgele konumda belirir
- Yem yenince yılan bir hücre büyür (kuyruğa yeni segment eklenir)

Temel Yapı:

```
1 HUCRE = 20 # piksel cinsinden hücre boyutu
2 COLS = 800 // HUCRE
3 ROWS = 600 // HUCRE
4
5 # Yilani liste olarak tut: [(sutun, satir), ...]
6 yilan = [(10, 10), (9, 10), (8, 10)]
7 yon = (1, 0) # (dx, dy): sag
8
9 # Her hareket adiminda:
10 yeni_bas = (yilan[0][0] + yon[0],
11             yilan[0][1] + yon[1])
12 yilan.insert(0, yeni_bas) # Basa yeni konum ekle
13 # Yem yenmediyse kuyruğu kaldır:
14 yilan.pop()
```

Adımlar:

1. Izgara arka planini ciz (isteğe bagli)
2. Yilani yeşil kareler olarak ciz
3. Yemi kırmızı kare olarak ciz
4. Basa girdi isleme ekle (K_UP, K_DOWN, K_LEFT, K_RIGHT)
5. Yeni bas yeme esitse: `yan.pop()` çağırma ve yeni yem oluştur
6. Basi duvara veya kuyruğa carpiyorsa oyunu sıfırla

İpucu: Yılan hizini kontrol etmek için her N karede bir adım atmak üzere bir sayac kullanın:

```
1 ADIM_KAREDE = 10 # kac karede bir adım atilacagi
2 sayac = 0
3
4 # Dongu icinde:
5 sayac += 1
6 if sayac >= ADIM_KAREDE:
7     sayac = 0
8     # Yilani hareket ettir
```

Başarılar!